

Register Allocation - II

Nikhil Hegde

Compiler Optimizations in LLVM
Lecture series @ QUALCOMM Inc.

Register Allocation - Goals

1. Minimize spill code.
 - If you need to spill, try to spill in the lesser executed part of the code.
2. Use as less registers as possible
3. Eliminate copy instructions

Register Allocation in LLVM - Passes

- Live Variables
- Phi Elimination
- Two address insertion
 - $A = B \text{ op } C \Rightarrow \text{mov } B \text{ } A; A = A \text{ op } C$
 - Why do this? Some targets require some arithmetic to be in this "A = A op C" form. `add rax rbx;`
- Slot Indexes
- Live Intervals
- Register Coalescer
- Machine Scheduler
- Greedy Allocator
- Register rewriter

* Transformation Pass Analysis Pass

Register Allocation in LLVM – Related Passes

- Virtual register rewriter
- Block Live-in lists
- Stack slot coloring
 - Provide precise information for live-interval queries.
How? Assign indices to each instruction.
- LICM
- Shrink Wrapping
 - Move prolog/epilog code nearer to the point of their use
- Prolog/Epilog Inserter

* Transformation Pass Analysis Pass

Two Address Instruction - Commutativity

```
%0 = COPY $r0  
%1 = COPY $r1  
%2 = ADD r0 r1  
$r1 = COPY %2
```



```
%0 = COPY $r0  
%1 = COPY $r1  
$r0 = ADD $r0 $r1  
$r1 = COPY $r0
```

```
%0 = COPY $r0  
%1 = COPY $r1  
%2 = ADD r1 r0  
$r1 = COPY %2
```



```
%0 = COPY $r0  
%1 = COPY $r1  
$r1 = ADD r1 r0  
$r1 = COPY %2
```

- The commutative nature of operations can be exploited to optimize and avoid COPY.
- Execute `make_twoaddr` to observe rewriting of an instruction in MIR with a two address instruction.

Register Coalescing

- Eliminate as many COPYs as possible
- But you can have more COPYs inserted later due to live range splitting (execute `make regallocator` and observe the file `elimphi_after_greedy.mir`)
- Execute `make coalescer` from code examples and observe the changes in the `.mir` files generated before and after.
- Execute `make dumpregalloc` and observe the dump file generated to see the detailed activities of the register coalescer.

Slot Indexes

- Data structure (mapping from instruction to a number)
 - For representing live intervals

```
0B  bb.0.entry:
    liveins: $xmm0, $xmm1
16B  %1:fr32 = COPY $xmm1
32B  %2:fr32 = COPY $xmm0
48B  MOVSSmr %stack.0.a.addr, 1, $noreg, 0, $noreg, %2:fr32 :: (store (s32) into %i
64B  MOVSSmr %stack.1.b.addr, 1, $noreg, 0, $noreg, %1:fr32 :: (store (s32) into %i
96B  %2:fr32 = nopexcept DIVSSrr %2:fr32(tied-def 0), %1:fr32, implicit $mxcsr
112B  $xmm0 = COPY %2:fr32
128B  RET 0, $xmm0
```

XMM0 [0B,32r:0)[112r,128r:1) 0@0B-phi 1@112r

Slot Indexes

XMM0 [0B,32r:0)[112r,128r:1) 0@0B-phi 1@112r

- XMM0 is live at two intervals [0B,32r:0) and [112r,128r:1)
- 0 and 112 are the index attached to instructions
- B = Boundary of an instruction
- r = read slot. Can be use/def.
- :0 / :1 = value numbering for XMM0
- 0@0B-phi = the value 0 at index 0B comes from a phi node.
And the value 1 at 112 is from a read slot

Data Structures used

- Priority Queue for managing slot indexes
- B+ tree for dynamically querying interference
- Neither graphs nor coloring on those

Virtual Register Rewriter

- Happens after Register Allocator
- Results are stored in `VirtRegMap`
 - Input for the next phase: Block Live-in Lists. These capture global liveness information.
 - `LiveRegUnits` and `LivePhysRegs` are passes to compute liveness information inside the basic block.

Prolog/Epilog Insertion Pass

- Callee Saved registers are stored / restored
PrologEpilogInsertionPass
- Shrink Wrapping decides the placement
- Register Scavenging: if extra registers are needed after register allocation (mini register allocator)
 - All uses must be in same basic block

Register Allocators in LLVM

- greedy
 - Default. Priority-based linear-scan with region splitting & live-range splitting.
- Basic
 - Linear scan allocation
- Fast
 - Assigns the first available register, spills aggressively. Use when compile time is more important.
- PBQP (partitioned Boolean Quadratic Programming)
 - Solves the graph coloring problem using PBQP

Question

- You are moving from a system with no cache memory (Cray-1) to a system with cache memory. Tell whether Register Allocation is “more useful”, “less useful”, or “does not matter” when you move.
- You are moving from a system with no cache memory (Cray-1) to a system with cache memory. Tell whether Instruction Scheduling is “more useful”, “less useful”, or “does not matter” when you move.

Question

- You are moving from a system with N registers to a system with $N/2$ registers. Tell whether Register Allocation is “more useful”, “less useful”, or “does not matter” when you move.
- You are moving from a system with N registers to a system with $N/2$ registers. Tell whether Instruction Scheduling is “more useful”, “less useful”, or “does not matter” when you move.

Question

- You are moving from a system with fast clock rate to a slow clock rate. Now all memory accesses take 1 cycle. Earlier memory took 100, cache took 50, registers took 1 cycle. Tell whether Register Allocation is “more useful”, “less useful”, or “does not matter” on this new system